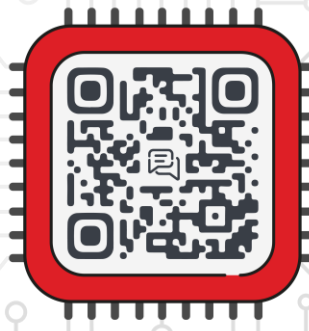




6

20

3000 sp

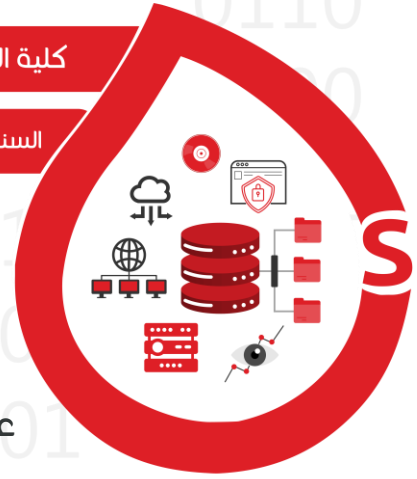
عملي

كلية الهندسة المعلوماتية

السنة الرابعة - قسم البرمجيات

Fragmentation & Join

عملي مشترك



RB Informatcs; 1/12/2025

قواعد معطيات ٢

ما هي ال Fragments ؟
ليكن لدينا مثلا الجداول الآتية:

Page 1		
ID	Name	Phone Number
1	A	1513
2	B	1217
4	D	1313

Page 2		
ID	Name	Phone Number
5	E	1515
6	F	1678
7	G	1567

وأردنا عمل INSERT لل 3 وكانت جميع الجداول ممتلئة ماذا سيحصل؟

حسب الترتيب في ال Index الذي لدينا يجب أن توضع بين ال 2 وال 4 ولكن page1 ممتلئة بالكامل.

أذاً سنقوم بعمل newpage بين ال page1 وال page2 ونضع فيها ال 4 ونقوم بوضع ال 3 في الجدول حسب الترتيب المناسب وتصبح ال page1 تؤثر على ال new page وال new page تقوم بالتأشير على ال page 2 وهذا هو مبدأ ال Fragmentation. أي ستصبح لدينا بهذا الشكل:

Page 1			New Page			Page 2		
ID	Name	Phone Number	ID	Name	Phone Number	ID	Name	Phone Number
1	A	1513	4	D	1313	5	E	1515
2	B	1217				6	F	1678
3	C	...				7	G	1567

وفي هذه الحالة يصبح لدينا إما صفحات ممثلة بالكامل أو صفحات تحتوي شواغر.

■ سؤال هل سيؤثر الأداء لدينا؟

لنقوم ببناء جدول بسيط يحتوي Non-Clustered Index:

```
CREATE TABLE TEST (  
  Id INT PRIMARY KEY,  
  Name VARCHAR(100),  
  email VARCHAR(200)  
)
```

```
CREATE NONCLUSTERED INDEX X_Test ON TEST(email)
```

أصبح لدينا ال Non-Clustered Index مبني على ال email.
ولاختبار عمل ال fragmentation سنقوم بتطبيق الكود الآتي:

```
DECLARE @id INT  
DECLARE @max INT  
DECLARE @name VARCHAR(100)  
  
SET @id = 1  
SET @max = 10000  
WHILE (@id <= @max)  
BEGIN  
  SET @name = CAST(LEFT(NEWID(), 60) AS VARCHAR(60))  
  INSERT INTO TEST (Id, Name, email)  
  SELECT @id, @name, @name + '@abc.com'  
  
  SET @id = @id + 1  
END  
  
SELECT * FROM test
```

- فهرس ال Non Clustered لا يغير ترتيب الصفوف في الجدول بل يخزن صفحة مرتبة من القيم لتسريع الوصول
- Declare @id : رقم السجل الحالي.
- @max : تعريف متغير.
- @id = 1 : تعريف متغير.
- @max=10000 : عدد السجلات المطلوب إدخالها.
- @name : اسم عشوائي لكل سجل.
- حلقة ال While: كل دورة يتم توليد اسم عشوائي باستخدام ال new id (اسم عشوائي فريد)
- حال سجل جديد في الجدول (TEST) ويحتوي على :
- ID رقم السجل.
- Name: الاسم العشوائي.
- Email: الاسم + "@abc.com".
- ويستمر حتى يتم إدخال ال 10000 سجل.
- SELECT * FROM TEST عرض البيانات (عرض كل البيانات الموجودة في السجل)

أي الهدف من هذا الكود :

- إدخال كمية ضخمة من البيانات بشكل سريع.
- اختبار الفهرسة على الأداء.
- مراقبة ال Fragment عن طريق الإدخال المتكرر.

نلاحظ أنه لدينا ال id يزداد بمقدار 1 ثم توليد ال name في ال new id الذي سيعطينا قيم عشوائية

عند قيامنا بأول إضافة يكون ال id=1 وسيأخذ كل من ال name وال email قيمةً عشوائية. في ال record الثاني يصبح لدينا ال id=2 وأيضاً كل من ال name وال email يأخذ قيمةً عشوائية. نستنتج أنه من غير الممكن أن يحدث Fragmentation على ال 10,000 سجل.

■ لماذا؟

- لأنه يتولد بشكل مرتب.

نلاحظ أن ال email يحمل قيمةً عشوائية عند كل Insert، لذلك حتماً سيحدث لدينا fragmentation على ال non-clustered index، لأنه عند كل insert يقوم ال optimizer بتعبئة ال pages بقيم عشوائية. مثلاً، عند امتلاء أول ثلاث صفحات وحصول record جديد وكانت قيمته في ال page 1، ماذا سيقوم بفعله؟ سيقوم بتوليد new page ليضع ال data في المكان المناسب، وذلك بسبب ال fragmentation. في مثالنا، نسبة ال fragmentation هي 99٪ وهي سيئة جداً.

- سؤال: لماذا يكون لدينا عدد الصفحات في الـ clustered index أكبر من عدد الصفحات في الـ non-clustered index بالرغم من أن الـ fragmentation قد تتم على الـ non-clustered index؟
- لأن الـ leaf node في الـ clustered index يحتوي على row كامل على عكس الـ leaf node في الـ non-clustered index الذي يحمل فقط الـ Emp id, Sales id.

فمن الطبيعي أن يكون عدد الصفحات في clustered index أكبر.

لنقم الآن بعمل الـ update:

```
UPDATE TEST
SET
    name = SUBSTRING(name, 1, 20),
    email = CAST(NEWID() AS VARCHAR(60)) + LEFT(email, CHARINDEX('@', email))
WHERE
    id BETWEEN 10000 AND 50000;
```

الـ name قمنا بتكبيره (قمنا بإضافة 20 حرف) وقمنا بتوليد قيم جديدة للـ id وإضافتها.

- سؤال: هل ما زال لدينا fragmentation على الـ non-clustered بعد القيام بالـ update؟ نعم.
- سؤال: هل أصبح الـ fragmentation على الـ clustered index (PK) بعد القيام بالـ update؟ نعم، لأنه سابقاً لاحظنا أن الـ fragmentation كانت 99٪ وقمنا بتكبير الـ data الخاصة بالاسم، فلن نستطيع تخزين الـ data بدون القيام بهذه العملية.
- لأن جميع الـ pages كانت ممتلئة أصلاً، فيقوم بعمل new page وسيوزع الـ data عليه.
- سؤال: هل يوجد طريقة لتخفيف أعباء الـ fragmentation؟ نعم، عن طريق عملية الـ REBUILD.
- هي عملية إعادة بناء الـ index من جديد، مما يضمن ترتيب الـ pages وتنظيمها.

في مثالنا

```
ALTER INDEX X_Test ON TEST REBUILD.
```

كلما زادت الـ Fragmentation، تراجع الأداء.

لذلك نستخدم الـ Rebuild حتى تقوم بعملية تحسين للاستعلامات (Queries) وتخفيف حمل عبء الصفحات الإضافية.

مراجعة:

- عملية الـ Fragmentation تعني تفتت البيانات داخل الصفحات أو الفهارس، مما يؤدي إلى بطء في الخادم (Server) وبطء في الأداء، وزيادة في استهلاك الموارد.
- ماذا تعني وكيف تحدث؟ عندما يتم إدخال أو حذف أو تعديل البيانات بشكل متكرر، فإن ترتيب الصفحات التي تخزن البيانات يصبح غير منظم.
- هل لها أنواع؟ نعم، لها نوعان:
- Internal Fragmentation (التفتت الداخلي): يحدث عندما تحتوي صفحات البيانات على فراغات غير مستخدمة. مثال: صفحة ممثلة 100% بالبيانات، بعد تحذف بعض الصفوف تصبح ممثلة بنسبة 70%. هذا يجبر النظام على قراءة صفحات أكثر للحصول على نفس كمية البيانات.
- External Fragmentation (التفتت الخارجي): يحدث عندما تكون صفحات البيانات غير مرتبة على القرص. بدلاً من أن تكون متتالية، تصبح موزعة في أماكن متفرقة، مما يؤدي إلى بطء الوصول للبيانات ويزيد من عمليات الإدخال والإخراج (I/O).

لماذا تعتبر مشكلة؟

- تستهلك ذاكرة ومعالجاً أكثر.
- تؤدي إلى بطء في الاستعلامات.
- تؤثر على أداء الفهارس وعمليات النسخ الاحتياطي.

كيف نحلها؟

باستخدام عملية الـ Rebuild.

JOIN

تذكرة INNER JOIN ويكتب بالشكل التالي:

```
SELECT _ , _
FROM _ INNER JOIN _
ON _ = _
```

مثال

```
SELECT t.title, p.pub_name
FROM titles AS t
INNER JOIN publishers AS p
ON t.pub_id = p.pub_id
```

ترجع السجلات من الجدولين جنباً إلى جنب، والتي يربط بينها الشرط الموجود في عبارة ON.
لتحقيق الـ JOIN بين الجداول، لدينا ثلاثة أنواع (يختارها الـ Optimizer):

1. Loop Join
2. Merge Join
3. Hash Join

والطرق الثلاث مرتبة حسب الأفضلية، وعلى الرغم من أن Loop Join هي الأفضل، إلا أن Hash Join هي الأكثر استخداماً.

لنقم بإنشاء جدولين
الجدول الأول: جدول الأقسام

```
CREATE TABLE dbo.dept(  
    DeptID INT IDENTITY(1,1) PRIMARY KEY,  
    DName VARCHAR(50) NOT NULL  
);  
  
INSERT INTO dbo.dept VALUES ('admin');  
INSERT INTO dbo.dept VALUES ('sales');  
INSERT INTO dbo.dept VALUES ('archive');
```

الجدول الثاني: جدول الموظفين

```
CREATE TABLE dbo.emp (  
    EmpID INT IDENTITY(1,1) PRIMARY KEY,  
    EName VARCHAR(50) NOT NULL,  
    Gender CHAR(1) NOT NULL,  
    Salary INT NOT NULL,  
    DeptID INT FOREIGN KEY REFERENCES dbo.dept(DeptID),  
    CONSTRAINT CHK_Emp_Salary CHECK (Salary > 0)  
);
```

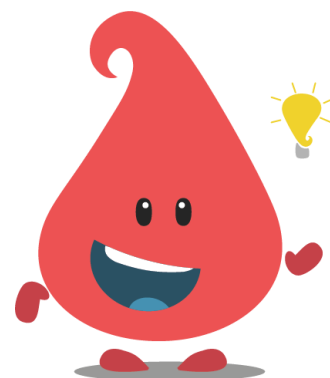
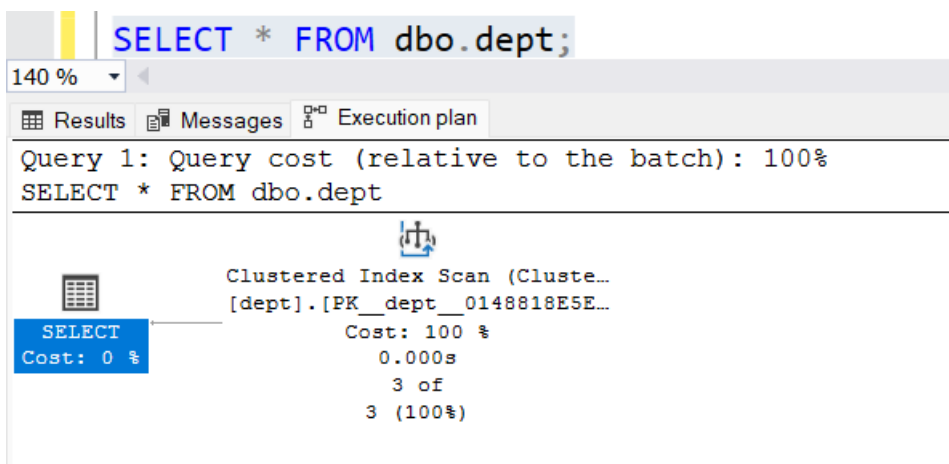
```

INSERT INTO dbo.emp VALUES ('Ahmad', 'M', 25000, 3);
INSERT INTO dbo.emp VALUES ('Sally', 'F', 50000, 2);
INSERT INTO dbo.emp VALUES ('Mirna', 'F', 80000, 1);
INSERT INTO dbo.emp VALUES ('Yazan', 'M', 25000, 3);
INSERT INTO dbo.emp VALUES ('Hadi', 'M', 50000, 2);

```

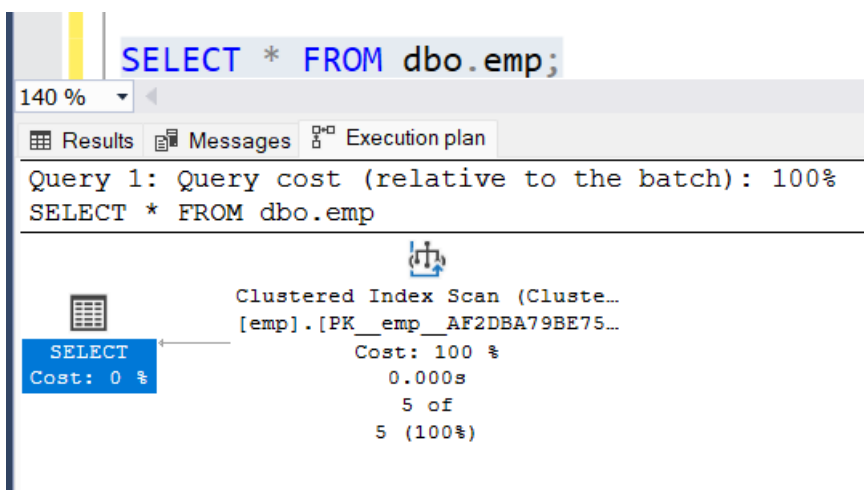
إذا قمنا بعمل الاستعلام على الجدول التالي:

```
SELECT * FROM dbo.dept;
```



وإذا قمنا بعمل الاستعلام على الجدول الثاني

```
SELECT * FROM dbo.emp;
```



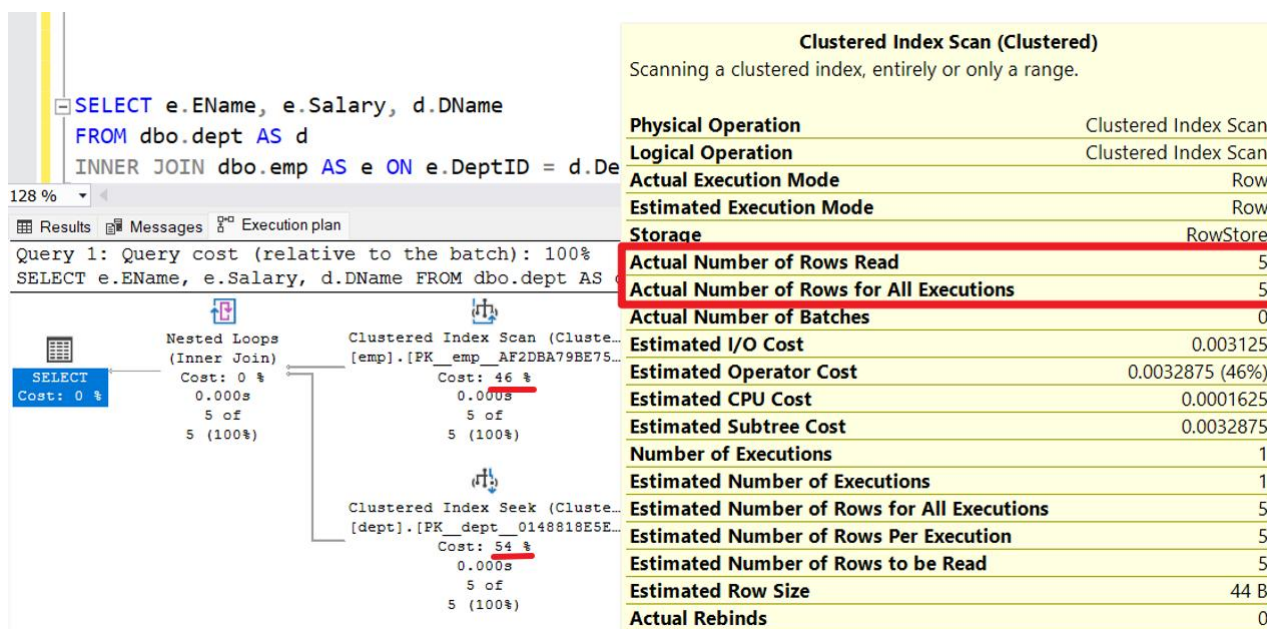
ويأتي المرض ليذكرنا بنعمة الصحة التي كثيراً ما ننسى حمد الله عليها

Loop Join

وهنا أيضًا، لنقم بعمل الاستعلام الآتي:

```
SELECT e.ENAME, e.Salary, d.DName
FROM dbo.dept AS d
INNER JOIN dbo.emp AS e ON e.DeptID = d.DeptID;
```

نلاحظ أن الـ optimizer يبدأ بالقراءة من جدول الـ emp وهو الـ outer table وينفذ عملية الجلب لمرة واحدة. وبهذه العملية قام بجلب الـ 10 أسطر (الأسطر التي قمنا بإضافتها).
إذا قمنا بفتح الـ output list نلاحظ أن ما تم جلبه يمثل جدول الـ emp، ونلاحظ أنه جلب الـ Salary والـ EName كما تم جلب الـ DeptID.



لاحظ أن

- %46 Clustered Index Scan
- %54 Clustered Index Seek

- ليتمكن من جلب اسم القسم عن طريقه، ولهذا السبب قام الـ optimizer بجلب البيانات من الـ outer table ثم ينتقل للـ inner table وهو الأسهل في التقييم.
- وبالتزامن مع جلب الـ rows من جدول الـ emp، يتم عمل الـ looping لكل سطر لجلب اسم القسم الخاص بالموظف من جدول الـ dept عن طريق الـ DeptID.
- وعملية جلب البيانات من جدول الـ dept يتم تنفيذها بالتساوي مع عدد الأسطر التي تم جلبها من جدول الـ emp.
- ونسمي جدول الـ outer table وجدول الـ inner table، ويتم تنفيذ الـ inner join بعدد السجلات التي تم جلبها نتيجة تنفيذ الـ outer.

```
SELECT e.EName, e.Salary, d.DName
FROM dbo.dept AS d
INNER JOIN dbo.emp AS e ON e.DeptID = d.DeptID;
```

154 %

Results Messages Execution plan

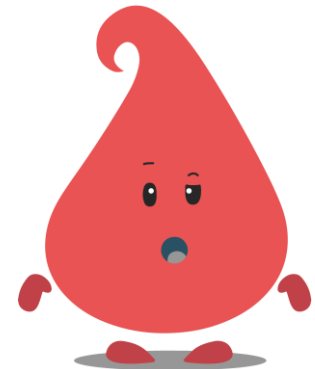
Query 1: Query cost (relative to the batch): 100%

SELECT e.EName, e.Salary, d.DName FROM dbo.dept AS d INNER JOIN dbo.emp AS e ON



- Clustered index scan => outer
- Non clustered index seek => inner

Nested Loops	
For each row in the top (outer) input, scan the bottom (inner) input, and output matching rows.	
Physical Operation	Nested Loops
Logical Operation	Inner Join
Actual Execution Mode	Row
Estimated Execution Mode	Row
Actual Number of Rows for All Executions	5
Actual Number of Batches	0
Estimated Operator Cost	0.0000209 (0%)
Estimated I/O Cost	0
Estimated CPU Cost	0.0000209
Estimated Subtree Cost	0.0072239
Number of Executions	1
Estimated Number of Executions	1
Estimated Number of Rows for All Executions	5
Estimated Number of Rows Per Execution	5
Estimated Row Size	67 B
Actual Rebinds	0
Actual Rewinds	0
Node ID	0



نلاحظ nested loop تم تنفيذها لمرة واحدة وتم جلب 5 اسطر تحوي على ال full query result set

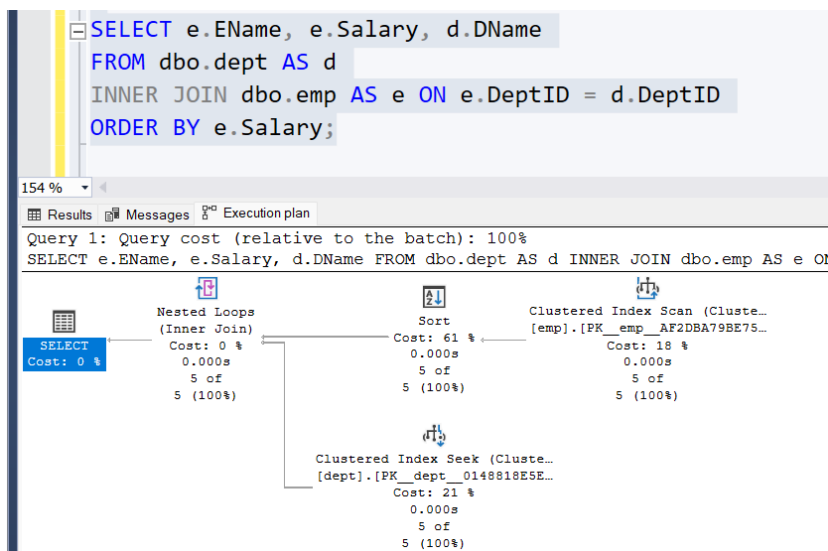
ملخص

كيفية عمل ال Loop Join تبدأ من جدول ال outer table لجلب السجلات، ثم لكل سطر يتم جلب السجل المقابل له من ال inner table.

لنعيد الاستعلام مع وضع ORDER BY :

```
SELECT e.ENAME, e.Salary, d.DName
FROM dbo.dept AS d
INNER JOIN dbo.emp AS e ON e.DeptID = d.DeptID
ORDER BY e.Salary;
```

نلاحظ أن الـ optimizer يبدأ بجلب الـ Salary و الـ EName والـ DeptID من جدول الـ emp (outer table). وقبل جلب الـ d.DName قام بعمل Sort، والسبب أن حجم البيانات في السجلات أصغر من حجم الـ result set النهائية، وبالتالي يكون الترتيب أسرع وأسهل.

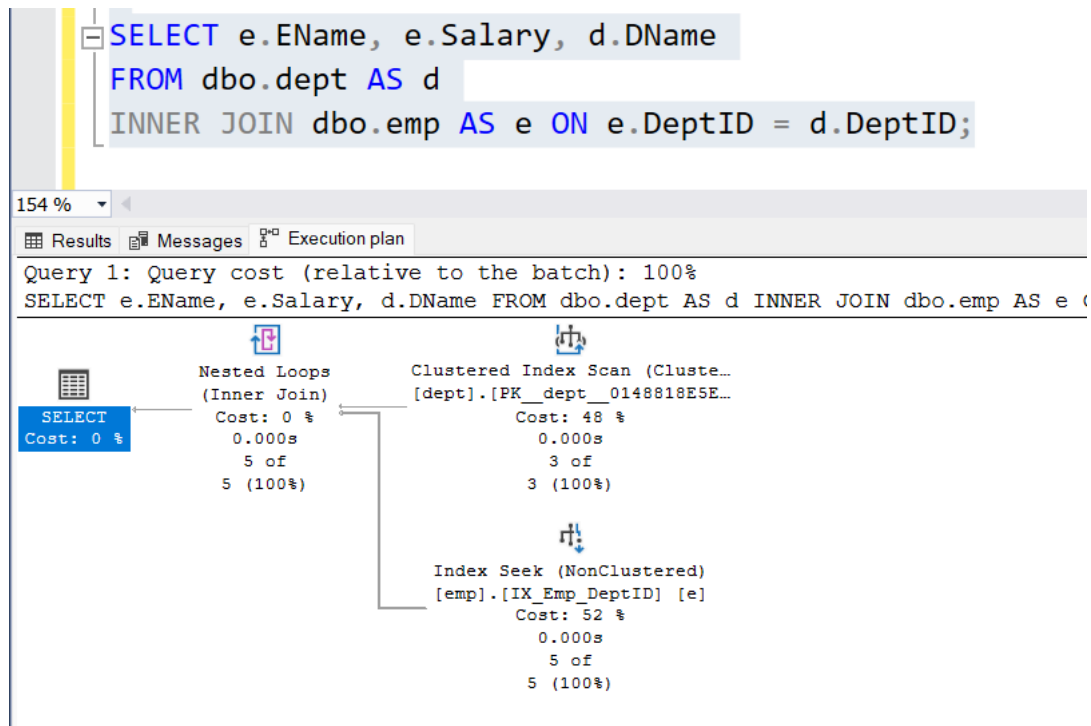


لنقم ببناء non-clustered index على الجدول emp :

```
CREATE NONCLUSTERED INDEX IX_Emp_DeptID ON dbo.emp (DeptID)
INCLUDE (ENAME, Salary);
```

والآن لننفذ الاستعلام مرة أخرى:

```
SELECT e.ENAME, e.Salary, d.DName
FROM dbo.dept AS d
INNER JOIN dbo.emp AS e ON e.DeptID = d.DeptID;
```

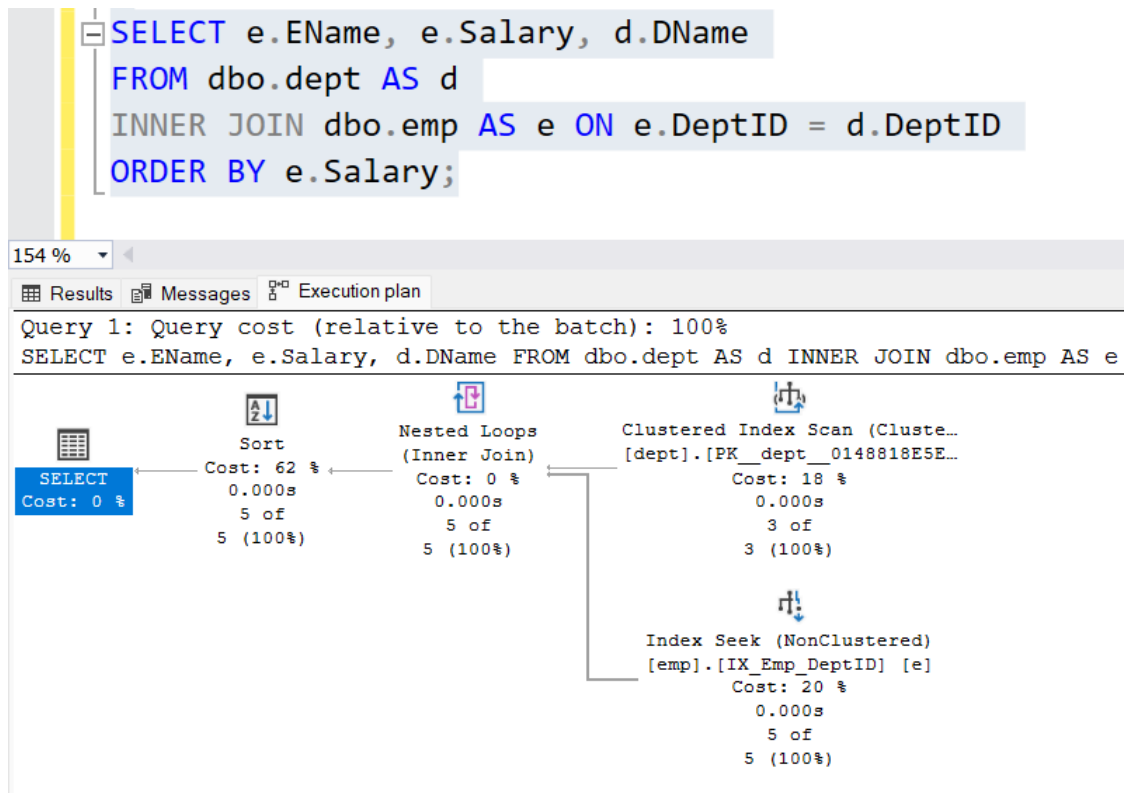


- حين يبدأ الـ engine بالتنفيذ، يلاحظ وجود الـ non-clustered index على جدول الـ emp والذي يحتوي على الـ DeptID (كمفتاح) مع INCLUDE للـ Salary و EName. وهو كل ما يحتاجه الاستعلام من جدول الـ emp.
- بما أن حجم السجل في الـ non-clustered index أصغر من حجم السجل في الـ clustered index في جدول الـ emp، ولأن هذا الـ index يغطي (covers) الأعمدة المطلوبة، يقرر الـ optimizer أن يبدأ بالمسح (Scan) من هذا الـ index الأصغر حجماً.
- عن طريق الـ DeptID الذي هو المفتاح (Key) في الـ non-clustered index، يقوم بعمل Seek لجلب البيانات المطابقة من جدول الـ dept، وبذلك يكون قد اختصر عملية الجلب ولم يحتاج إلى جلب عمود Gender لأنه غير مطلوب في الاستعلام.

إضافة ORDER BY مرة أخرى:

```
SELECT e.ENAME, e.SALARY, d.DNAME
FROM dbo.dept AS d
INNER JOIN dbo.emp AS e ON e.DeptID = d.DeptID
ORDER BY e.SALARY;
```

نلاحظ أنه تم إضافة عامل Sort مرة أخرى. والسبب هو أن عملية الترتيب (ORDER BY) تتم على عمود Salary من جدول الـ emp، بينما عملية الجلب الأساسية بدأت من جدول الـ dept (الجدول الخارجي في هذه الحالة). لذلك، لا يمكنه ترتيب النتائج مباشرة ويجب أن ينتظر حتى يتم جمع جميع البيانات ثم يقوم بعملية الترتيب النهائية.



Merge Join

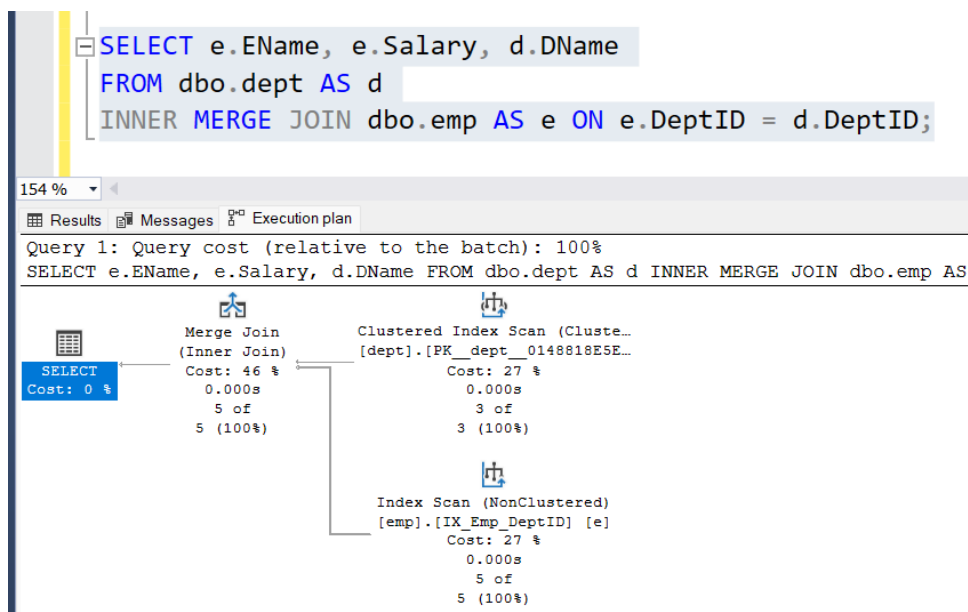
تعتمد على البحث عن مدخلين (inputs) لهما نفس الترتيب ثم عمل مقارنة بينهما.
شرح:

لنقم بعمل hint لإجبار الـ optimizer على التنفيذ بطريقة الـ Merge Join.

```

SELECT e.ENAME, e.Salary, d.DName
FROM dbo.dept AS d
INNER MERGE JOIN dbo.emp AS e ON e.DeptID = d.DeptID;

```



■ كيف يعمل الـ Merge Join ؟

■ يبدأ الـ Merge Join بالبحث عن فهرسيين (من الجدولين dept و emp) مرتبين بنفس الترتيب حسب عمود الربط (DeptID).

لدينا في مثالنا

جدول الـ dept يوجد عليه clustered index مرتب بناءً على الـ DeptID.
وجداول الـ emp يملك non-clustered index على الـ DeptID (الذي قمنا بإنشائه في المثال السابق).
إذاً، يوجد مدخلان (inputs) مرتبان بنفس التسلسل، فيقوم الـ optimizer بعمل مقارنة بينهما بعد المسح.
بإضافة ORDER BY :

```
SELECT e.ENAME, e.Salary, d.DName
FROM dbo.dept AS d
INNER MERGE JOIN dbo.emp AS e ON e.DeptID = d.DeptID
ORDER BY e.Salary;
```

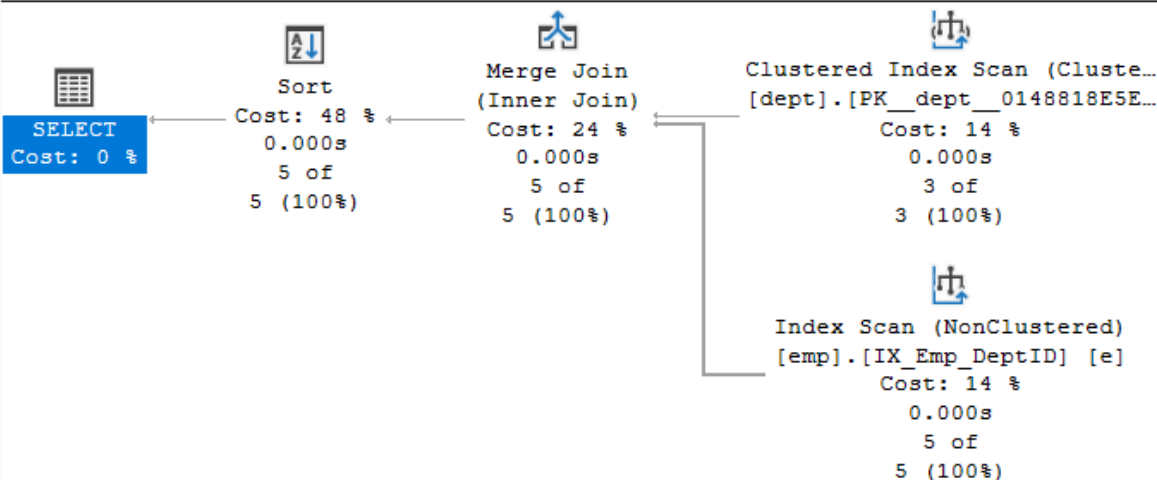
```
SELECT e.ENAME, e.Salary, d.DName
FROM dbo.dept AS d
INNER MERGE JOIN dbo.emp AS e ON e.DeptID = d.DeptID
ORDER BY e.Salary;
```

154 %

Results Messages Execution plan

Query 1: Query cost (relative to the batch): 100%

SELECT e.ENAME, e.Salary, d.DName FROM dbo.dept AS d INNER MERGE JOIN dbo.emp AS

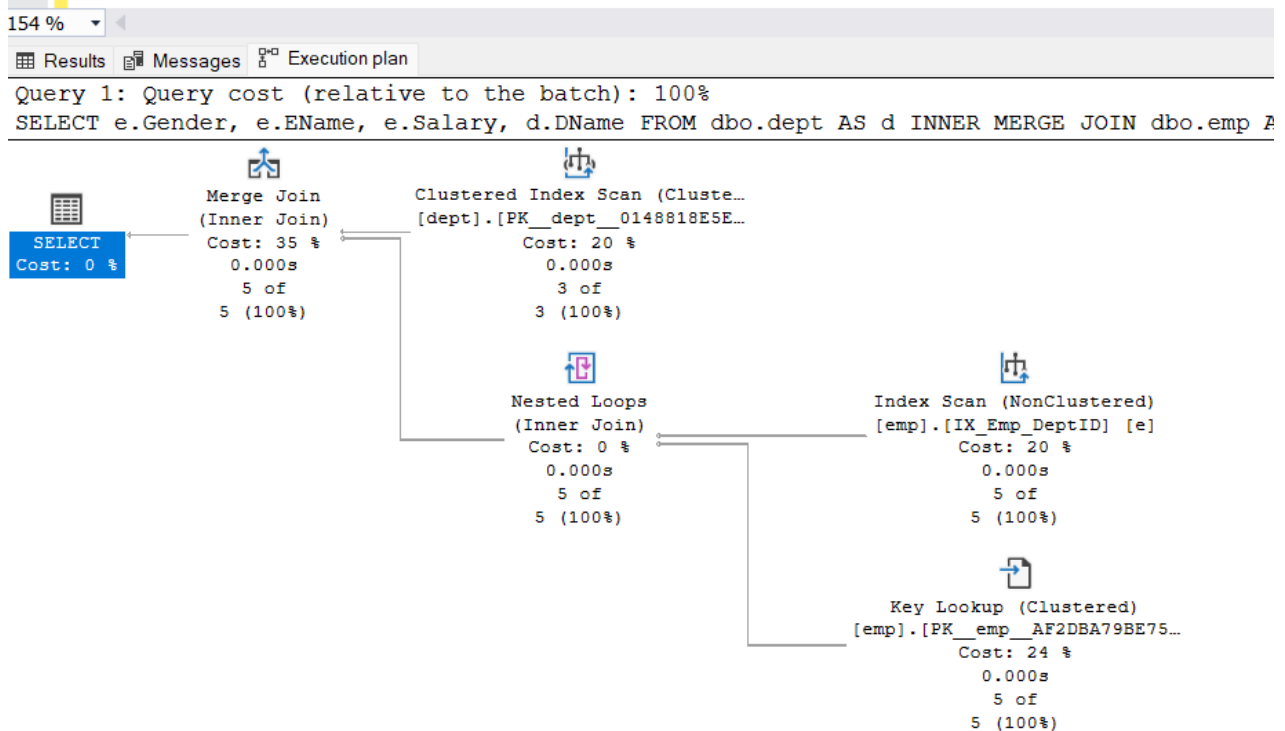


يقوم بعمل Sort بعد ال Merge. والسبب أن ال result set الناتج عن عملية الدمج تكون مرتبة أصلاً حسب ال DeptID. ليتمكن من عمل المقارنة، ولذلك يضطر ال optimizer للانتظار حتى انتهاء عملية الدمج ثم يقوم بإعادة ترتيب النتائج حسب ال Salary.

عند إضافة عمود Gender:

```
SELECT e.Gender, e.EName, e.Salary, d.DName
FROM dbo.dept AS d
INNER MERGE JOIN dbo.emp AS e ON e.DeptID = d.DeptID;
```

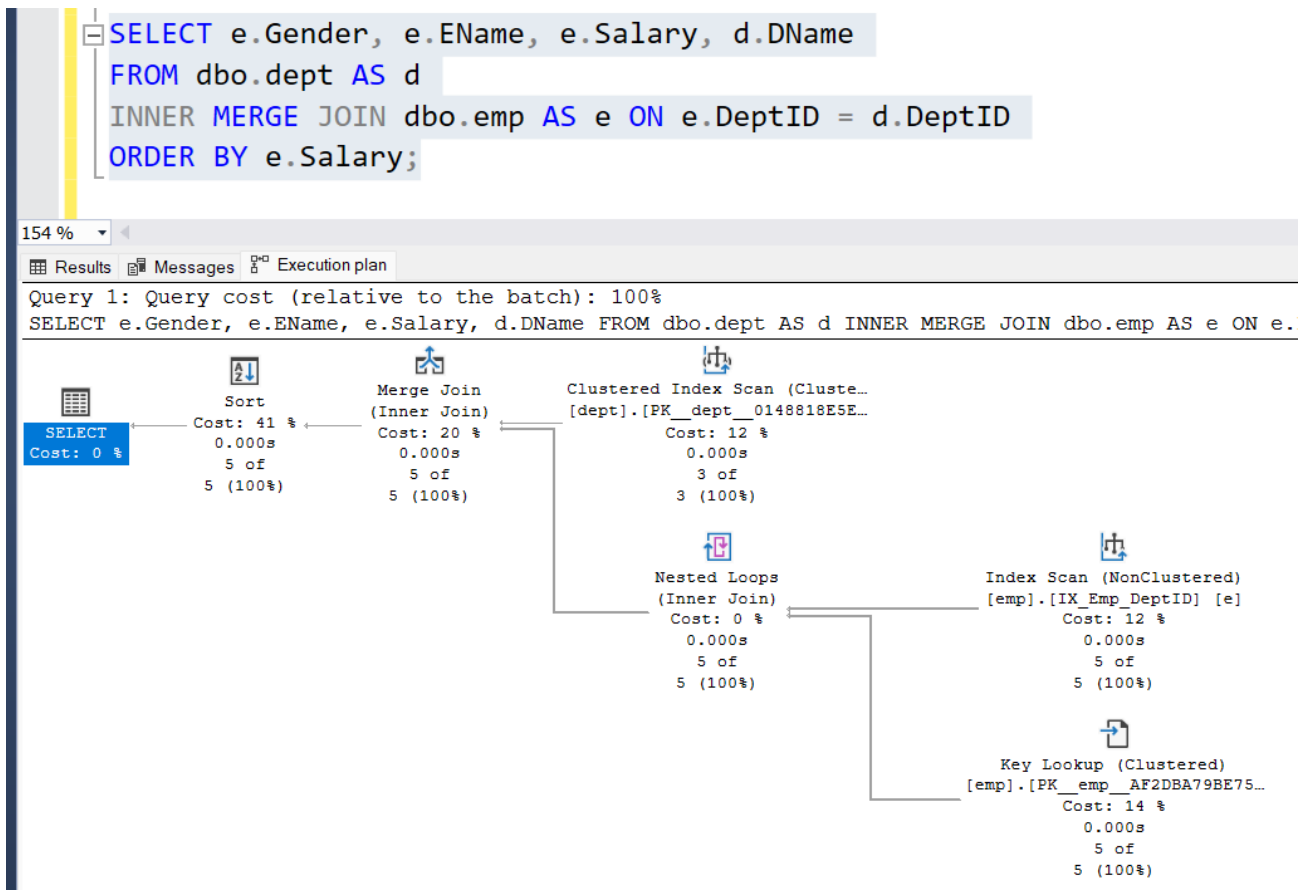
```
SELECT e.Gender, e.EName, e.Salary, d.DName
FROM dbo.dept AS d
INNER MERGE JOIN dbo.emp AS e ON e.DeptID = d.DeptID;
```



نلاحظ، أنه لم يعد من الممكن تنفيذ ال Merge Join بالاعتماد على ال non-clustered index (IX_Emp_DeptID) وحده، لأنه لا يحتوي على عمود ال Gender. في هذه الحالة، سيقوم ال optimizer بعمل Key Lookup (بحث باستخدام المفتاح) على ال clustered index لجدول ال emp لجلب قيمة ال Gender لكل سجل، ثم يكمل عملية الدمج.

لنقم بإضافة ORDER BY على الاستعلام السابق:

```
SELECT e.Gender, e.EName, e.Salary, d.DName
FROM dbo.dept AS d
INNER MERGE JOIN dbo.emp AS e ON e.DeptID = d.DeptID
ORDER BY e.Salary;
```



يتم عمل Sort بعد الدمج، وذلك لأن عملية الدمج تتطلب أن يكون المدخلان (inputs) مرتبين بنفس التسلسل حسب الـ DeptID لإتمام المقارنة، وبعد ذلك يتم إعادة ترتيب المجموعة النهائية من البيانات حسب الـ Salary.

- سؤال في حال لم يكن جدول الـ emp يملك non-clustered index مرتب بناءً على الـ DeptID، وقمنا بعمل hint لإجبار الـ optimizer على عمل الـ Merge Join، فما الذي سيحصل؟
- سيقوم الـ optimizer بإضافة عامل Sort لجدول الـ emp لترتيبه حسب عمود الـ DeptID قبل البدء بعملية الدمج. وهذا ما يفسر لماذا قد ترى أحياناً أن البيانات تأتي مرتبة في خطة التنفيذ حتى لو لم يكن هناك ORDER BY في الاستعلام.

Hash Join

يتم تطبيقه عندما تكون الاستعلامات معقدة جداً أو تحوي على الكثير من INNER JOIN و subquery. وفي هذه الحالة، لا يعود الـ optimizer قادراً على دمج (merge) الـ result sets ببعضها البعض بكفاءة لأن حجمها كبير جداً في الذاكرة.

■ كيف جاءت فكرة الـ Hashing؟

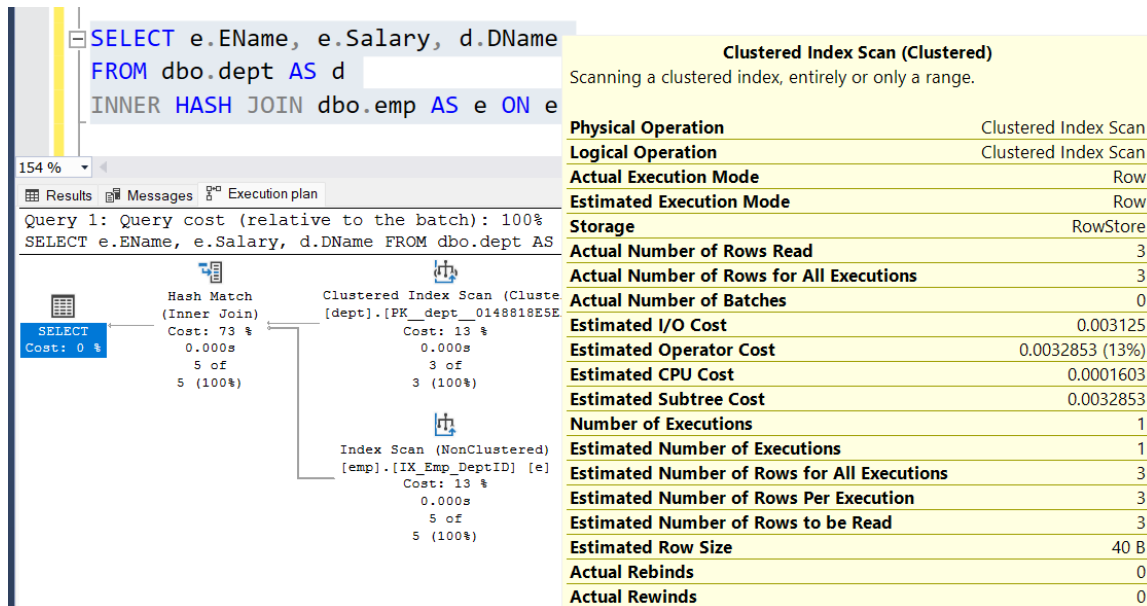
1. يأخذ الـ engine مجموعة الـ values من الجدول الأول حسب الـ join key.
2. يمرر هذه الـ values لدالة (function) ترجع قيمة مشفرة (hash value).
3. يتم تخزين هذه القيم في جدول تجزئة (hash table) في الذاكرة.
4. يتم تنفيذ نفس الدالة على الجدول الثاني.
5. السجلات التي ترجع نفس القيمة المشفرة تكون مطابقة للسجلات في الجدول الأول.

```
ELECT e.ENAME, e.SALARY, d.DNAME
FROM dbo.dept AS d
INNER HASH JOIN dbo.emp AS e ON e.DeptID = d.DeptID;
```

Clustered Index Scan (Clustered)	
Scanning a clustered index, entirely or only a range.	
Physical Operation	Clustered Index Scan
Logical Operation	Clustered Index Scan
Actual Execution Mode	Row
Estimated Execution Mode	Row
Storage	RowStore
Actual Number of Rows Read	3
Actual Number of Rows for All Executions	3
Actual Number of Batches	0
Estimated I/O Cost	0.003125
Estimated Operator Cost	0.0032853 (13%)
Estimated CPU Cost	0.0001603
Estimated Subtree Cost	0.0032853
Number of Executions	1
Estimated Number of Executions	1
Estimated Number of Rows for All Executions	3
Estimated Number of Rows Per Execution	3
Estimated Number of Rows to be Read	3
Estimated Row Size	40 B
Actual Rebinds	0
Actual Rewinds	0
Ordered	False
Node ID	1

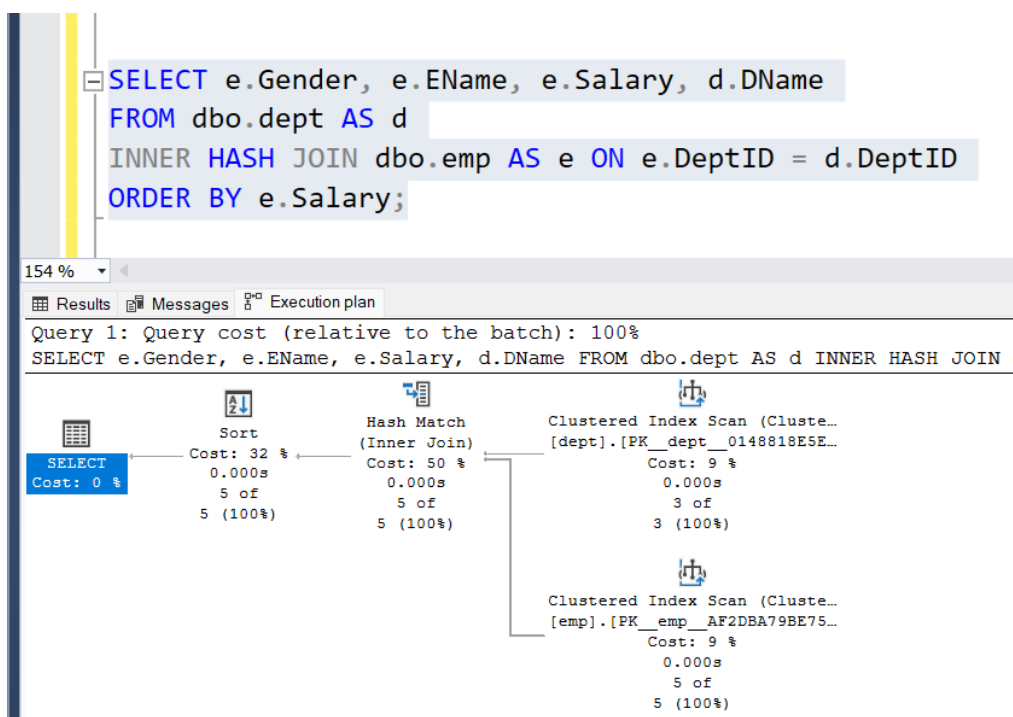
نلاحظ أنه يتم تنفيذ المسح (Scan) مرة واحدة وجلب 3 سجلات من جدول الـ dept.

```
SELECT e.ENAME, e.Salary, d.DName
FROM dbo.dept AS d
INNER HASH JOIN dbo.emp AS e ON e.DeptID = d.DeptID;
```



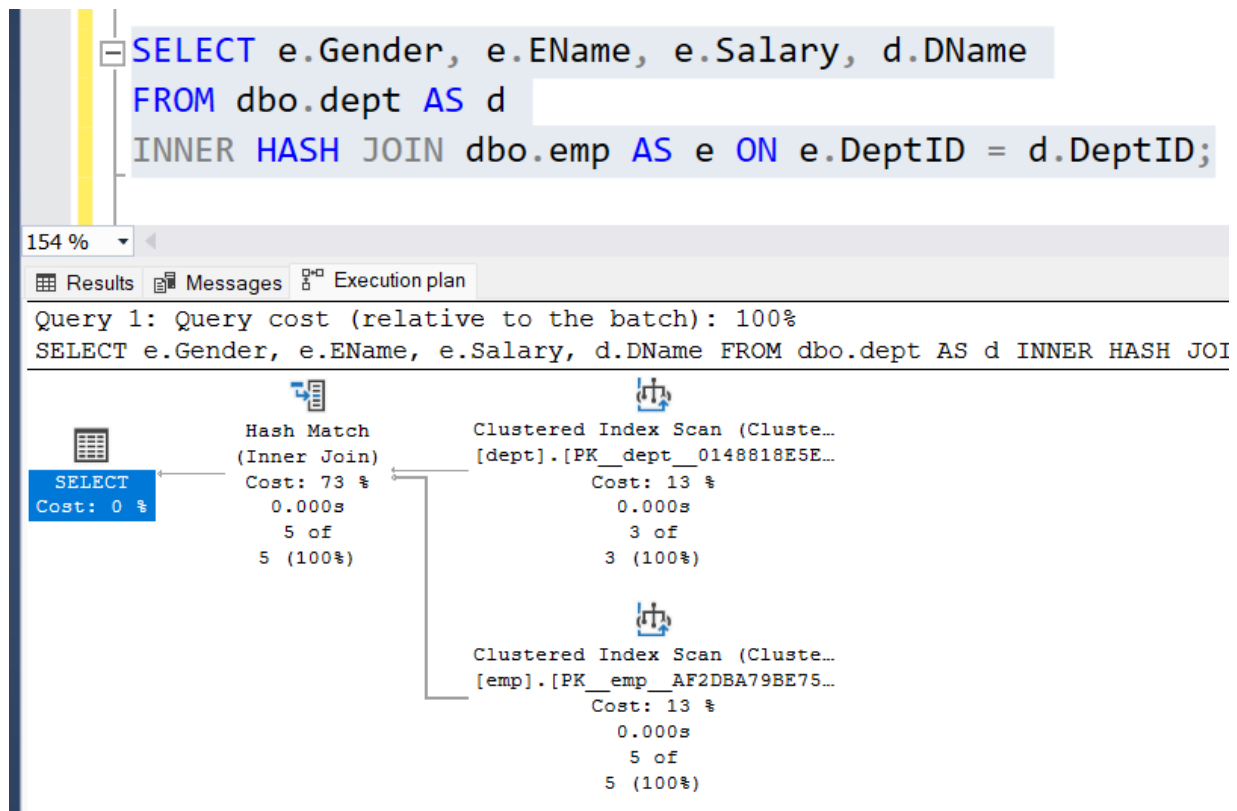
نلاحظ أنه قام بالتنفيذ لمرة واحدة وجلب 5 سجلات من جدول الـ emp. كيف تم ذلك؟ عن طريق الـ hash key وهو الـ DeptID في كلا الجدولين لدينا، فإذا قمنا بإضافة ORDER BY:

```
SELECT e.Gender, e.ENAME, e.Salary, d.DName
FROM dbo.dept AS d
INNER HASH JOIN dbo.emp AS e ON e.DeptID = d.DeptID
ORDER BY e.Salary;
```



نلاحظ أنه قام بعمل الـ Hashing ثم الـ Sort، لأنه من غير المنطقي ترتيب البيانات أثناء عملية الـ Hashing نفسها.
إضافة عمود Gender لجملته SELECT:

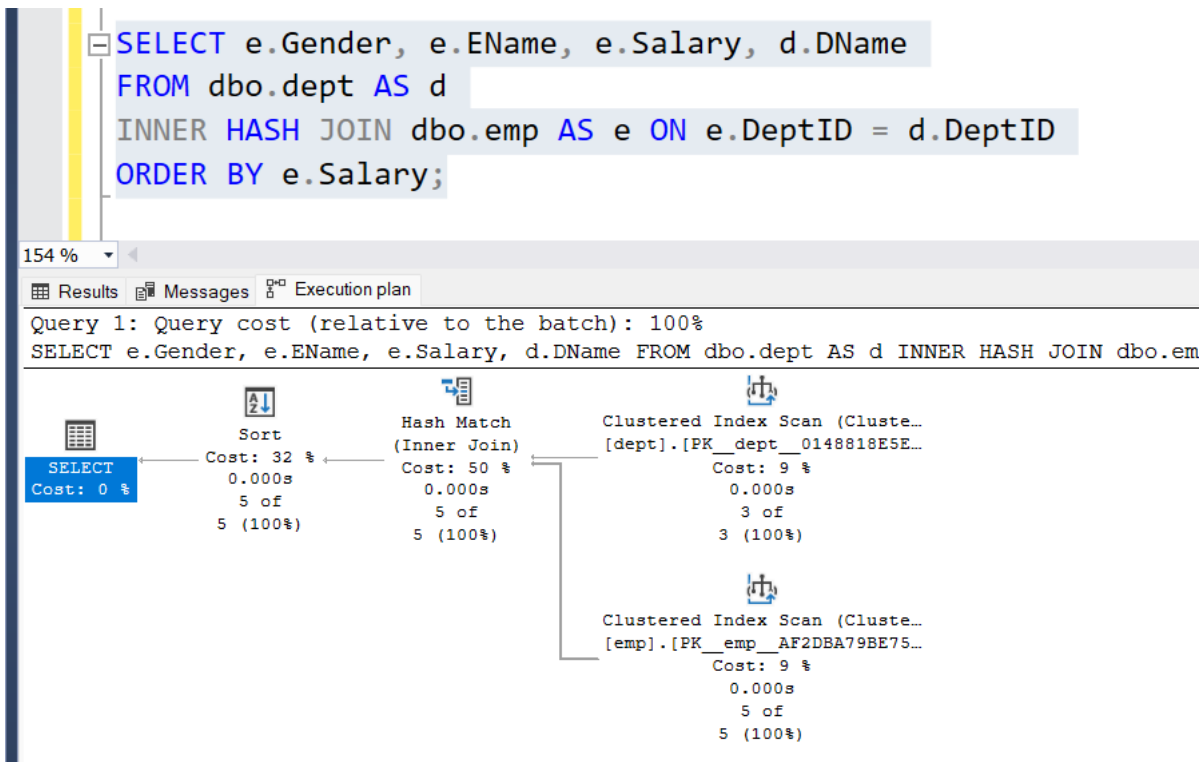
```
SELECT e.Gender, e.ENAME, e.Salary, d.DName
FROM dbo.dept AS d
INNER HASH JOIN dbo.emp AS e ON e.DeptID = d.DeptID;
```



نلاحظ أن الـ Hash Join في هذه الحالة قد يختار تنفيذ Key Lookup لاستكمال البيانات (مثل Gender)، لأنه وجد أن هذا الأسلوب أسهل من مسح (Scan) جدول أكبر ومن ثم معالجته، وبشكل عام يعتبر الـ Sort من أسوأ العمليات تكلفة.

إضافة ORDER BY مرة أخرى:

```
SELECT e.Gender, e.ENAME, e.Salary, d.DName
FROM dbo.dept AS d
INNER HASH JOIN dbo.emp AS e ON e.DeptID = d.DeptID
ORDER BY e.Salary;
```



نلاحظ أن التنفيذ لم يختلف كثيراً، وتم عمل Sort في النهاية بعد عمل الـ Hash. ودائماً يتم عمل الـ Sort بعد الـ Hash عند وجود ORDER BY على أعمدة ليست هي مفتاح الـ Join.

ملخص لأنواع الـ Join

1. Loop Join

متى يستخدم:

- عندما يكون أحد الجدولين صغيراً، أو عندما يكون الجدول الخارجي صغيراً ويوجد فهرس مناسب على الجدول الداخلي.
- كيف يعمل:
- يمر على كل صف من الجدول الخارجي (Outer Table) ويبحث عن الصفوف المطابقة في الجدول الداخلي (Inner Table).
- ملاحظات:
- بسيط وسريع في الحالات الصغيرة.
- بطيء جداً إذا كان الجدولان كبيرين بدون فهرس مناسبة.

2. Merge Join

- متى يستخدم:
- عندما يكون الجدولان مرتبين مسبقاً (عند وجود فهرس مرتب على أعمدة الربط في كلا الجدولين).
- كيف يعمل:
- يمر على الجدولين بترتيب ويقارن الصفوف (يشبه دمج قائمتين مرتبتين).
- ملاحظات:
- سريع جداً مع البيانات المرتبة.
- قد يتطلب عملية Sort إذا لم تكن البيانات مرتبة، مما يضيف تكلفة.

3. Hash Join

- متى يستخدم:
- في الاستعلامات الكبيرة أو المعقدة، وعندما لا توجد فهرس مناسبة أو ترتيب يساعد في أنواع الـ Join الأخرى.
- كيف يعمل:
- ينشئ جدول تجزئة (hash table) في الذاكرة من أحد الجدولين (الـ Build input)، ثم يبحث عن المطابقات من الجدول الآخر (الـ Probe input).
- ملاحظات:
- فعال جداً مع البيانات الكبيرة وغير المرتبة.
- يستهلك ذاكرة أكثر من الأنواع الأخرى.

التعقيد	الأداء مع البيانات الكبيرة	يحتاج فهرس	الأفضلية في	
بسيط	ضعيف	نعم (يفضل)	جداول صغيرة	Nested Loop Join
متوسط	ممتاز	نعم	بيانات مرتبة	Merge Join
أعلى	جيد جداً	لا	بيانات غير مرتبة	Hash Join

النهاية نلتقاكم بالمحاضرة القادمة